

MWORKS 专业培训系列（一）

Syslab中的数学运算和基础使用方法

孔德才

苏州同元软控信息技术有限公司

2025年4月2日

目录

1. 常用操作命令和快捷键

2. 矩阵分析

3. 脚本文件和函数文件

4. 与MATLAB的差异小结

1. 常用操作命令和快捷键

命令	功能	命令	功能
Ctrl+L/clc()	清空命令行窗口	save	保存内存变量
clear()	清除工作区变量	load	加载制定文件中的变量
clf()	清除图形窗口	hold("on")	图形保持命令
println	显示变量或文字的内容	cd/pwd/mv	更改当前文件夹/显示当前文件夹/移动或重命名文件或文件夹

使用无参数函数，需要加上对应的括号

```
julia> a = [1 2 3; 4 5 6; 7 8 10]
3×3 Matrix{Int64}:
 1  2  3
 4  5  6
 7  8 10

julia> clear()

julia> a
```

```
for i in 1:5
    println(i)
end
```

```
p = rand(1, 10)
q = ones(10)
filename = "test.mat"
save(filename, "-mat"; p, q)
```

```
julia> load("test.mat")
Dict{String, Any} with 2 entries:
 "q" => [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
 "p" => [0.553362 0.246858 ... 0.0099528 0.0955967]
```



1. 常用操作命令和快捷键

按钮和快捷键	功能	按钮和快捷键	功能	按钮和快捷键	功能
↑	调用上一行	Ctrl + ←	光标左移一个单词	Alt+←	清除当前输入字段
↓	调用下一行	Ctrl + →	光标右移一个单词	Del	删除光标处字符
←	光标左移一个字符	Home/Ctrl+a	光标置于当前行开头	Backspace	删除光标前字符
→	光标右移一个字符	End	光标置于当前行结尾	Ctrl + D	终止程序

代码补全

```
julia> mid[Tab]
#julia> middle
```

利用代码补全功能查询函数的使用方法

```
julia> sum(
```

```
julia> sum(
sum(f, x::FillArrays.AbstractFill) @ FillArrays C:\Users\Public\TongYuan\.julia\package
s\FillArrays\KpKaA\src\FillArrays.jl:496
sum(f::Union{Function, Type}, a::StaticArraysCore.StaticArray{<:Tuple, T}; dims) where
T @ StaticArrays C:\Users\Public\TongYuan\.julia\packages\StaticArrays\MSJcA\src\mapred
uce.jl:289
sum(f, a::StaticArraysCore.StaticArray{<:Tuple, T}; dims) where T @ StaticArrays C:\Use
rs\Public\TongYuan\.julia\packages\StaticArrays\MSJcA\src\mapreduce.jl:288
sum(r::OffsetArrays.OffsetVector{T, <:AbstractRange{T}} where T) @ OffsetArrays C:\User
s\Public\TongYuan\.julia\packages\OffsetArrays\hwmb\src\OffsetArrays.jl:594
sum(A::AbstractArray, w::StatsBase.AbstractWeights, dims::Int64) @ StatsBase deprecated
.jl:103
```

sum函数的各种使用方法（包含了函数或方法所在的模块和源码文件）



1. 常用操作命令和快捷键

系统预定义的特殊变量

符号	意义	无限精度值	Unicode 字符代码
pi/ π	圆周率	# 3.1415...	\pi
ans	结果显示默认变量名	\	\
e/exp()	自然常数 (v0.6 之前为 英文字母 e)	# 2.7182...	\euler
im	虚数单位	# real(im * im) == -1	

```
pi
# $\pi$  = 3.1415926535897...
eps()
#2.220446049250313e-16

#复数的定义
1+2im
e
#e = 2.7182818284590...
```

- 在 Syslab 和 MATLAB 中，变量 ans 被设置为交互式会话中提交的最后一个表达式的值
- 不同的是，当 Julia 代码以非交互式模式运行时并不会设置 ans



1. 常用操作命令和快捷键

设置数据输出格式 `ty_format("style",x)`

Style	结果	示例
default	Julia语言默认数值打印格式	π
short	短固定十进制小数点格式，小数点后包含 4 位数。这是默认数值设置	3.1416
long	长固定十进制小数点格式，Float64 值的小数点后包含 15 位数，Float32 值的小数点后包含 7 位数	3.141592653589793
shortE	短科学记数法，小数点后包含 4 位数	3.1416e+00
longE	长科学记数法，Float64 值的小数点后包含 15 位数，Float32 值的小数点后包含 7 位数	3.141592653589793e+00
shortG	短固定十进制小数点格式或科学记数法（取更紧凑的一个），总共4位	3.142
longG	长固定十进制小数点格式或科学记数法（取更紧凑的一个），对于 Float64 值，总共 15 位；对于 Float32 值，总共 7 位	3.141592653589793e+00
shortENG	短工程记数法，小数点后包含 4 位数，指数为 3 的倍数	3.1416E0
longENG	长工程记数法，包含 15 位有效位数，指数为 3 的倍数	3.14159265358979E0
+	正/负格式，对正、负和零元素分别显示 +、- 和空白字符	+
bank	货币格式，小数点后包含 2 位数	3.14
hex	二进制双精度数字的十六进制表示形式	400921fb54442d18
rat	小整数的比率	165707065//52746197

不得复印、传播或以其他形式使用

```
julia> pi
π = 3.1415926535897...

julia> ty_format("default",pi)
π

julia> ty_format("short",pi)
3.1416

julia> ty_format("long",pi)
3.141592653589793
```



目录

1. 常用操作命令和快捷键

2. 矩阵分析

3. 脚本文件和函数文件

4. 与MATLAB的差异小结

2.1 矩阵构造-直接输入

建立矩阵： 将矩阵的元素用中括号 `[]` 括起来，按照矩阵行的顺序输入各个元素，同一行的各元素之间用空格分隔，不同行的元素之间用分号或 `alt+enter` 分隔

常用构造方式如下：

- `A=[a b;c d]`
- `A=[a b`
`c d]`

$a = [1 \ 2 \ 3]$

```
julia> a = [1 2 3]
1×3 Matrix{Int64}:
 1  2  3
```

$a = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

```
julia> b = [1 2;3 4]
2×2 Matrix{Int64}:
 1  2
 3  4
```

方式1

$a = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$

```
julia> a1=[1;2;3;4;]
4×1 Matrix{Int64}:
 1
 2
 3
 4
```

```
julia> b = [1 2
            3 4]
2×2 Matrix{Int64}:
 1  2
 3  4
```

方式2



2.1 矩阵构造-特殊矩阵

一些常见的特殊矩阵构造

函数	描述
<code>zeros(T, dims)</code>	每个元素均为0的Array
<code>ones(T, dims)</code>	每个元素均为1的Array
<code>eye(T,dims)</code>	单位矩阵
<code>true(dims)</code>	每个元素均为true的BitArray
<code>false(dims)</code>	每个元素均为false的BitArray
<code>rand(T, dims)</code>	一个随机Array，元素值是[0,1)半开区间中均匀分布且服从一阶独立分布
<code>randn(T,dims)</code>	一个随机 Array，元素为标准正态分布，服从独立同分布
<code>fill(x, dims)</code>	一个被值x填充的Array

说明:

- T为数据类型
- dims为数组维度

```
julia> a=zeros(Int,3,3)#如果不定义数据类型，默认浮点型
3×3 Matrix{Int64}:
 0  0  0
 0  0  0
 0  0  0

julia> b=ones(3,3)
3×3 Matrix{Float64}:
 1.0  1.0  1.0
 1.0  1.0  1.0
 1.0  1.0  1.0

julia> c=eye(3,3)
3×3 Matrix{Int64}:
 1  0  0
 0  1  0
 0  0  1

julia> d=true(2,2)
2×2 BitMatrix:
 1  1
 1  1

julia> e=rand(2,3)
2×3 Matrix{Float64}:
 0.883481  0.0833875  0.797534
 0.133065  0.113993  0.24801

julia> f=fill(3,2,2)
2×2 Matrix{Int64}:
 3  3
 3  3
```

2.2 矩阵分析

Syslab中提供了大量关于**矩阵属性**、**线性方程**、**特征值与奇异值**、**矩阵分解**、**矩阵运算**、**矩阵结构**等不同类型的函数

类别	函数	功能描述
矩阵属性	norm	计算范数
	det	矩阵行列式
	rank	矩阵的秩
	tr	对角线元素之和
	cond	逆运算的条件数
	condskeel	相对条件数
	condeig	与特征值有关的条件数
	rref	简化的行阶梯形矩阵 (Gauss-Jordan 消元法)
	subspace	两个子空间之间的角度
线性方程	\	对线性方程组 $Ax = B$ 求解 x
	/	对线性方程组 $xA = B$ 求解 x
	linsolve	对线性方程组求解
	inv	矩阵求逆
	pinv	Moore-Penrose 伪逆
	lskov	存在已知协方差情况下的最小二乘解
	sylvester	求 Sylvester 方程 $AX + XB = C$ 的 X 解

类别	函数	功能描述
特征值与奇异值	eigvals	返回A的特征值
	eigen	计算 A 的特征值分解, 返回一个特征分解对象 F
	svd	计算 A 的奇异值分解 (SVD) 并返回一个 SVD 对象
	svdvals	按降序返回 A 的奇异值
	ordeig	拟三角矩阵的特征值
	polyeig	多项式特征值问题
	hessenberg	矩阵的 Hessenberg 形式
	schur	Schur 分解
	schur!	Schur 分解
	rsf2csf	将实数Schur形式变换为复数Schur形式
	cdf2rdf	拟三角矩阵的特征值
矩阵分解	lu	LU 矩阵分解
	ldlt	实对称三对角矩阵S的LDLt分解
	cholupdate	Cholesky 分解的秩 1 更新
	qrdelete	从 QR 分解中删除列或行
	qrinsert	将列或行插入 QR 分解
	qrupdate	QR 分解的秩 1 更新
	planerot	Givens 平面旋转

类别	函数	功能描述
矩阵运算	transpose	转置向量或矩阵
	' ,adjoint	复共轭转置
	sqrt	矩阵平方根
	exp	矩阵指数运算
	log	矩阵对数运算
	kron	Kronecker 张量积
	cross, x	叉积
	dot, ·	点积
矩阵结构	bandwidth	矩阵的上下带宽
	tril	矩阵的下三角形部分
	triu	矩阵的上三角形部分
	isbanded	确定矩阵是否在特定带宽范围内
	isdiag	确定矩阵是否为对角矩阵
	ishermitian	确定矩阵是 Hermitian 矩阵
	issymmetric	确定矩阵是对称矩阵
	istril	确定矩阵是否为下三角矩阵
	istriu	确定矩阵是否为上三角矩阵

2.3 矩阵属性

Syslab提供了描述矩阵特征（属性）的值的求解函数，如**矩阵的秩、迹、范数、条件数**

例：创建一个随机方阵，求解方阵对应行列式的值，矩阵的秩，矩阵的迹，矩阵的2-范数以及矩阵的条件数

```
#创建一个随机方阵A，方阵A里面的元素值介于1-3，维度是3x3的方阵
```

```
julia> A = randi(3, (3, 3))
```

```
3×3 Matrix{Int64}:
```

```
 3  1  1
 3  1  2
 2  2  1
```

```
#求解方阵A对应行列式的值
```

```
julia> det(A)
```

```
-4.0
```

```
#求解矩阵A的秩
```

```
julia> rank(A)
```

```
3
```

```
#求解矩阵A的迹，与matlab不同，matlab中为trace
```

```
julia> tr(A)
```

```
5
```

```
#求解矩阵A的2-范数
```

```
julia> norm(A)
```

```
5.830951894845301
```

```
#求解矩阵A的条件数
```

```
julia> cond(A)
```

```
9.085295841756352
```

注意：

矩阵范数具有不同的类型，若想得到不同的范数，可参考以下形式：

`norm(A, p)`

其中可根据p的不同取值，得到不同类型的范数结果，常见的几种范数如下：

```
#求解矩阵A的1-范数
```

```
julia> norm(A, 1)
```

```
16.0
```

```
#求解矩阵A的2-范数，等效于norm(A)
```

```
julia> norm(A, 2)
```

```
5.830951894845301
```

```
#求解矩阵A的无穷范数
```

```
julia> norm(A, Inf)
```

```
3.0
```

2.3 矩阵属性

例：创建一个 3×3 幻方矩阵。在矩阵的末尾添加一个额外的列，计算该矩阵的简化行阶梯形矩阵，并索引该额外（增广）列

```
#创建一个3x3的魔方矩阵A
julia> A = magic(3)
3×3 Matrix{Int64}:
 8  1  6
 3  5  7
 4  9  2

#对矩阵添加额外一列[1,1,1]，形成增广矩阵
julia> A = hcat(A, [1, 1, 1])
3×4 Matrix{Int64}:
 8  1  6  1
 3  5  7  1
 4  9  2  1

#计算 A 的简化行阶梯形矩
julia> R,= rref(A)
([1.0 0.0 0.0 0.06666666666666665;
 0.0 1.0 0.0 0.06666666666666667;
 0.0 0.0 1.0 0.06666666666666668], [1, 2, 3])

#对 R 进行索引该额外（增广）列
julia> x = R[:,end]
3-element Vector{Float64}:
 0.06666666666666665
 0.06666666666666667
 0.06666666666666668
```

注意：

- magic函数为构造特殊矩阵魔方矩阵的函数，其他特殊矩阵如范德蒙矩阵(vander)、希尔伯特矩阵(hilb)、伴随矩阵(compan)、帕斯卡矩阵(pascal)具有对应构造函数，详见帮助文档
- 代码中A = hcat(A,[1, 1, 1])为水平方向串联数组，为矩阵拼接的一种方式，等效于A =[A [1, 1, 1]]
- 矩阵拼接的其他函数还有：cat、vcat、hvcata、hvncat等，具体含义及使用方法可参见帮助文档
- rref函数结果包含两个部分，分别为简化行阶梯形的 A 以及非零主元列 p，若只想显示简化后的行阶梯矩阵则代码改为：R = rref(A)[1]
- 代码R[:,end]为索引矩阵R的最后一列的所有值，具体矩阵索引方法可参见帮助文档或《Julia基础语法数组索引与切片》章节内容



2.4 线性方程

为强调线性方程组求解和求逆计算之间的区别，Syslab引入了 “\” 和 “/”

1. 当线性方程的系数矩阵为方阵时，采用矩阵的逆运算或采用左除运算进行求解

- 采用逆运算：X=inv(A)*B
- 采用左除：X=A\B

例：求下列方程组的解 X=[x1;x2;x3]

$$x_1 + 2x_2 + 3x_3 = 5$$

$$x_1 + 4x_2 + 9x_3 = -2$$

$$x_1 + 8x_2 + 27x_3 = 6$$

说明：

- 在Syslab中，求方阵A的逆矩阵可调用函数inv(A)
- 从执行时间和数值准确性方面而言，一种更好的方法是使用矩阵反斜杠运算符，即 $x = A \backslash b$ 。这会使用高斯消去法求解，而不必显式构造逆矩阵
- Julia中：如A为奇异矩阵，则打印错误:SingularException(3)

```
julia> A = [1 2 3; 1 4 9; 1 8 27]
3×3 Matrix{Int64}:
 1  2  3
 1  4  9
 1  8 27

julia> B = [5, -2, 6]
3-element Vector{Int64}:
 5
-2
 6

julia> X = inv(A) * B
3-element Vector{Float64}:
 23.0
-14.5
 3.666666666666667

julia> X = A \ B
3-element Vector{Float64}:
 23.0
-14.5
 3.666666666666667
```

```
julia> A = [1 2 3; 4 5 6; 7 8 9]
3×3 Matrix{Int64}:
 1  2  3
 4  5  6
 7  8  9

julia> inv(A)
ERROR: SingularException(3)
```



2.4 线性方程

2. 当线性方程的系数矩阵不为方阵时，采用矩阵的伪逆运算或采用左除的运算进行求解

- 采用伪逆运算： $X = \text{pinv}(A) * B$
- 采用左除： $X = A \backslash B$

例：求下列方程组的解 $X = [x_1; x_2; x_3]$

$$6x_1 + 3x_2 + 4x_3 = 3$$

$$-2x_1 + 5x_2 + 7x_3 = -4$$

$$8x_1 - 4x_2 - 3x_3 = -7$$

$$x_1 + 5x_2 - 7x_3 = 9$$

说明：

- 在Syslab中，求矩阵A的伪逆矩阵可调用函数 $\text{pinv}(A)$
- 当矩阵A的逆不存在时，Syslab中 $A \backslash B$ 或 B/A 仍然合理，因为Syslab利用伪逆矩阵进行了求解， $A \backslash B$ 等于 $X = \text{pinv}(A) * B$
- 本例为超定方程组（方程个数大于未知数个数），对于不定方程组的求解类似（方程个数小于未知数个数），这里不做过多说明

```
julia> A = [6 3 4; -2 5 7; 8 -4 -3; 1 5 -7]
4×3 Matrix{Int64}:
 6   3   4
-2   5   7
 8  -4  -3
 1   5  -7
```

```
julia> B=[3,-4,-7,9]
4-element Vector{Int64}:
 3
-4
-7
 9
```

```
julia> X=A\B
3-element Vector{Float64}:
-0.15636859142946918
 1.0095243810740877
-0.6952221590715199
```

```
julia> X=pinv(A)*B
3-element Vector{Float64}:
-0.15636859142946902
 1.0095243810740877
-0.6952221590715204
```

2.4 线性方程

3. Syslab提供 `X = linsolve(A,B)` 使用以下方法之一求解线性方程组 $AX = B$:

- 当 A 是方阵时, `linsolve` 使用 LU 分解和部分主元消去法
- 对于所有其他情况, `linsolve` 使用 QR 分解和列主元消去法

例: 使用`linsolve`分别求解求下列方程组的解 $X=[x_1;x_2;x_3]$

$$x_1 + 2x_2 + 3x_3 = 5$$

$$x_1 + 4x_2 + 9x_3 = -2$$

$$x_1 + 8x_2 + 27x_3 = 6$$

$$x_1 + 2x_2 + 3x_3 = 5$$

$$x_1 + 4x_2 + 9x_3 = -2$$

$$2x_1 + 4x_2 + 6x_3 = 10$$

说明:

- 如果矩阵未正确缩放或接近奇异矩阵, `linsolve` 计算的数值将不准确。使用 `rcond` 或 `cond` 检查矩阵的条件数
- 矩阵对角线缩放可以使用 `balance` 函数

```
julia> A=[1 2 3;1 4 9;1 8 27]
3×3 Matrix{Int64}:
 1  2  3
 1  4  9
 1  8 27
julia> B=[5,-2, 6]
3-element Vector{Int64}:
 5
-2
 6
julia> x =linsolve(A,B)
3-element Vector{Float64}:
 23.0
-14.5
 3.6666666666666665
```

```
julia> A=[1 2 3;1 4 9;2 4 6]
3×3 Matrix{Int64}:
 1  2  3
 1  4  9
 2  4  6
julia> B=[5,-2,10]
2-element Vector{Int64}:
 5
-2
10
julia> x =linsolve(A,B)
3-element Vector{Float64}:
12.0
-3.5
-0.0
julia> x =rcond(A)
0.0
julia> x =cond(A)
3.265704774793407e17
```



2.5 特征值与奇异值-特征值与特征向量

A是一个 $n \times n$ 的矩阵，若存在非零向量 x 和实数 λ 使得 $Ax = \lambda x$ ，则 x 和 λ 分别为 A 的特征向量和对应的特征值
特征值的个数等于A的阶数

Syslab中，特征值函数为 `eigvals`，特征向量函数为 `eigvecs`

- `E=eigvals(A)`：求矩阵A的全部特征值，构成向量E
- `E=eigvecs(A)`：将返回矩阵A特征向量

例：定义A,B矩阵如下，求A,B增广矩阵的特征值与特征向量

$$A = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \quad B = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

```
julia> A = [1 0; 0 -1]
2×2 Matrix{Int64}:
 1  0
 0 -1

julia> B = [0 1; 1 0]
2×2 Matrix{Int64}:
 0  1
 1  0

julia> λ = eigvals(A, B)
2-element Vector{ComplexF64}:
 0.0 - 1.0im
 0.0 + 1.0im

julia> x = eigvecs(A, B)
2×2 Matrix{ComplexF64}:
 0.0+1.0im  0.0-1.0im
-1.0+0.0im -1.0-0.0im
```



2.5 特征值与奇异值-奇异值分解

Syslab提供了svd函数计算A的奇异值分解(SVD)并返回一个SVD对象

- `F=svd(A; full=true)`: 计算A的“满”奇异值分解
- `F=svd(A,B)`: 计算 A 和 B 的广义奇异值分解

例：求矩阵 A 的“满”奇异值分解以及 A、B 的广义奇异值分解

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \quad B = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 10 \end{bmatrix}$$

```
julia> A = [1 2 3; 4 5 6; 7 8 9]; B = [1 2 3; 4 5 6; 7 8 10]
3×3 Matrix{Int64}:
 1  2  3
 4  5  6
 7  8  9
```

```
julia> F = svd(A; full=true)
SVD{Float64, Float64, Matrix{Float64},
Vector{Float64}}
U factor:
3×3 Matrix{Float64}:
-0.214837  0.887231  0.408248
-0.520587  0.249644 -0.816497
-0.826338 -0.387943  0.408248
singular values:
3-element Vector{Float64}:
16.84810335261421
 1.06836951455471
 4.418424751193368e-16
Vt factor:
3×3 Matrix{Float64}:
-0.479671 -0.572368 -0.665064
-0.776691 -0.0756865  0.625318
-0.408248  0.816497 -0.408248
```

```
julia> F = svd(A,B)
GeneralizedSVD{Float64, Matrix{Float64},
Float64, Vector{Float64}}
U factor:
3×3 Matrix{Float64}:
 0.182574  0.894427  0.408248
 0.0       0.707107  0.0
 0.0       0.0      1.33227e-15
D2 factor:
3×3 Matrix{Float64}:
 0.377964  0.0      0.0
 0.0       0.707107  0.0
 0.0       0.0      1.0
R0 factor:
3×3 Matrix{Float64}:
-0.613469 -0.605292 16.4577
 0.0      -1.25373 -10.1404
 0.0       0.0    -14.5945
```



2.6 矩阵分解

矩阵分解是将矩阵拆解为数个矩阵的乘积，可分为三角分解、满秩分解、QR分解、Jordan分解、奇异值分解等

- 三角(LU)分解法
- 正交(QR) 分解法

1. 矩阵LU分解将一个矩阵表示为一个可交换下三角矩阵和一个上三角矩阵的乘积

(方阵A非奇异, $\det(A) \neq 0$, LU分解总是可以进行的)

例：创建一个3×3矩阵A，并计算LU因子

$$A = \begin{bmatrix} 10 & -7 & 0 \\ -3 & 2 & 6 \\ 5 & -1 & 5 \end{bmatrix}$$

```
julia> A = [10 -7 0; -3 2 6; 5 -1 5]
3×3 Matrix{Int64}:
 10  -7  0
  -3   2  6
   5  -1  5
```

```
julia> L, U = lu(A)
LU{Float64, Matrix{Float64}, Vector{Int64}}
L factor:
3×3 Matrix{Float64}:
 1.0  0.0  0.0
 0.5  1.0  0.0
-0.3 -0.04 1.0
U factor:
3×3 Matrix{Float64}:
10.0 -7.0 0.0
 0.0  2.5 5.0
 0.0  0.0 6.2

julia> L*U
3×3 Matrix{Float64}:
10.0 -7.0 0.0
 5.0 -1.0 5.0
-3.0  2.0 6.0

julia> L,U,P = lu(A)
LU{Float64, Matrix{Float64}, Vector{Int64}}
L factor:
3×3 Matrix{Float64}:
 1.0  0.0  0.0
 0.5  1.0  0.0
-0.3 -0.04 1.0
```

```
U factor:
3×3 Matrix{Float64}:
10.0 -7.0 0.0
 0.0  2.5 5.0
 0.0  0.0 6.2
```

```
julia> m=diagm([1,1,1])
3×3 Matrix{Int64}:
 1  0  0
 0  1  0
 0  0  1
```

```
julia> P=m[P,:]
3×3 Matrix{Int64}:
 1  0  0
 0  0  1
 0  1  0
```

```
julia> P'*L*U
3×3 Matrix{Float64}:
10.0 -7.0 0.0
-3.0  2.0 6.0
 5.0 -1.0 5.0
```

此处和MATLAB有点区别（排列顺序与MATLAB不一致），Syslab是利用MKL求解



2.6 矩阵分解

矩阵分解是将矩阵拆解为数个矩阵的乘积，可分为三角分解、满秩分解、QR分解、Jordan分解、奇异值分解等

- 三角(LU)分解法
- 正交(QR) 分解法

2. 矩阵QR分解将一个矩阵表示为一个正交矩阵Q和一个上三角矩阵R的乘积

调用格式: $[Q,R] = qr(A)$

例: 对五阶魔方矩阵进行QR因子分解

```
julia> A=magic(5)
5×5 Matrix{Int64}:
 17  24   1   8  15
 23   5   7  14  16
  4   6  13  20  22
 10  12  19  21   3
 11  18  25   2   9
```

```
julia> qr(A)
LinearAlgebra.QRCompactWY{Float64, Matrix{Float64}, Matrix{Float64}}
Q factor:
5×5 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}:
-0.523387  0.505754  0.67347  -0.121542  -0.0441038
-0.708111 -0.696573 -0.0177274  0.0815416 -0.0800023
-0.12315  0.136742 -0.355751 -0.630744 -0.664635
-0.307875 0.191057 -0.412231 -0.424672  0.720021
-0.338662 0.451439 -0.499631  0.632767 -0.177441
R factor:
5×5 Matrix{Float64}:
-32.4808 -26.6311 -21.3973 -23.7063 -25.8615
 0.0      19.8943  12.3234  1.94393  4.08558
 0.0      0.0     -24.3985 -11.6316 -3.74148
 0.0      0.0      0.0     -20.0982 -9.97394
 0.0      0.0      0.0      0.0     -16.0005

julia> qr(A).Q
5×5 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}:
-0.523387  0.505754  0.67347  -0.121542  -0.0441038
-0.708111 -0.696573 -0.0177274  0.0815416 -0.0800023
-0.12315  0.136742 -0.355751 -0.630744 -0.664635
-0.307875 0.191057 -0.412231 -0.424672  0.720021
-0.338662 0.451439 -0.499631  0.632767 -0.177441
```



2.7 总结

Syslab帮助文档中提供关于**矩阵运算和分析**的所有函数说明以及示例

Syslab使用手册

筛选

文档主页

按分类

√ 文档980

√ 基础函数库407

语言基础97

数学292

图形12

图像6

所有407 文档416 示例0 函数0 App0

搜索的是 矩阵

1 2 3 4 ... 41 共 407 条

矩阵和数组 - 数组的创建、合并、重构、重新排列以及索引 数组是 Syslab 中信息和数据的基本表示形式。您可以构造常用的数组和网络、操作数组的形状和内容、访问数组元素，以及广播和矢量化函数，使其不仅能应用于标量，还能应用于数组中的每一个元素。
矩阵和数组 Matrix 生成 矩阵 eye 单位 矩阵 blkdiag 分块对角 矩阵 ty_hcat 空集和 矩阵 (向量) 的横向拼接 ty_vcat 空集和 矩阵 (向量) 的纵向拼接 ...
文档 > 基础函数库 > 语言基础 > 矩阵和数组

稀疏矩阵 - 初等稀疏 矩阵、重新排序算法、迭代法、稀疏线性代数 使用稀疏 矩阵 可极大地减少存储数据所需的内存量。
稀疏 矩阵 初等稀疏 矩阵、重新排序算法、迭代法、稀疏线性代数 使用稀疏 矩阵 可极大地减少存储数据所需的内存量。 spalloc 为稀疏 矩阵 分配空间 spzeros 创建长度为 m 的稀疏向量或大小为 m x n 的稀疏 矩阵 ...
使用 Syslab > 基础函数库 > 数学 > 稀疏矩阵

*** - 矩阵乘法**
矩阵乘法 $C = AB$ 是 A 和 B 的 矩阵 乘积。如果 A 是 $m \times p$ 矩阵，B 是 $p \times n$ 矩阵，则 C 是通过以下公式定义的 $m \times n$ 矩阵：对于非标量 A 和 B，A 的列数必须等于 B...
使用 Syslab > 基础函数库 > 数学 > 线性代数 > 矩阵运算 > *

*** - 矩阵乘法**
矩阵乘法 $C = AB$ 是 A 和 B 的 矩阵 乘积。如果 A 是 $m \times p$ 矩阵，B 是 $p \times n$ 矩阵，则 C 是通过以下公式定义的 $m \times n$ 矩阵：对于非标量 A 和 B，A 的列数必须等于 B...
使用 Syslab > 基础函数库 > 数学 > 初等数学 > 算术运算 > 乘法 > *

polyvalm 矩阵多项式计算
矩阵多项式计算 $Y = \text{polyvalm}(p,X)$ 以 矩阵 方式返回多项式 p 的计算值。此计算方法等同于使用多项式 p 替换 矩阵 X。示例 特征多项式的 矩阵 计算 求解 4 阶帕斯卡 矩阵 的特征多项式。 ...
使用 Syslab > 基础函数库 > 数学 > 初等数学 > 多项式 > polyvalm



目录

1. 常用操作命令和快捷键

2. 矩阵分析

3. 脚本文件和函数文件

4. 与MATLAB的差异小结

3.1 脚本文件与函数文件

Syslab中，jl文件是一系列代码组成的一个扩展名为.jl的文件

- jl文件可分为脚本文件（Syslab scripts）和函数文件（Syslab functions）
- 脚本文件是包含多条Syslab命令的文件；函数文件包含输入变量，并把结果传送给输出变量

```
FunTest.jl x vanderpoldemo.jl
1 # 初始值问题
2 using TyMath
3 using TyPlot
4 #直接写函数
5 # function vanderpoldemo(t,y,Mu)
6 #     return [y[2]; Mu*(1-y[1]^2)*y[2]-y[1]]
7 # end
8
9 tspan = [0 20];
10 y0 = [2; 0];
11 Mu = 1;
12 #调用函数
13 include("vanderpoldemo.jl")
14 ode = (t,y)->vanderpoldemo(t,y,Mu);
15 t,y = ode45(ode, tspan, y0);
16
17 plot(t,y[:,1])
18 xlabel("t")
19 ylabel("solution y")
20 title("van der Pol Equation,  $\mu = 1$ ")
```

```
FunTest.jl vanderpoldemo.jl x
1 function vanderpoldemo(t,y,Mu)
2     return [y[2]; Mu*(1-y[1]^2)*y[2]-y[1]]
3 end
```

```
# 初始值问题
using TyMath
using TyPlot
#直接构造函数
# function vanderpoldemo(t,y,Mu)
#     return [y[2]; Mu*(1-y[1]^2)*y[2]-y[1]]
# end
tspan = [0 20];
y0 = [2; 0];
Mu = 1;
#调用函数
include("vanderpoldemo.jl")
ode = (t,y)->vanderpoldemo(t,y,Mu);
t,y = ode45(ode, tspan, y0);

plot(t,y[:,1])
xlabel("t")
ylabel("solution y")
title("van der Pol Equation,  $\mu = 1$ ")
```

```
function vanderpoldemo(t,y,Mu)
    return [y[2]; Mu*(1-y[1]^2)*y[2]-y[1]]
end
```

关于函数文件名：函数文件名与函数名可以相同或不相同

3.2 函数文件注释

为 Julia 函数添加文档注释

- 在 Julia 中，可以使用三重引号 ("""") 和 Markdown 语法为函数添加文档字符串
- 文档字符串通常放在函数定义之前，Julia 的文档系统会自动识别并解析文档字符串
- 在 REPL 中使用 `?` 进入帮助模式可以查看函数的文档

```
"""  
##  my_sum  
  
my_sum(first, last)  
  
计算 [first, last] 之间的所有整数的求和  
"""  
function my_sum(first::Integer, last::Integer)  
    if first > last  
        return 0  
    end  
    return div((first + last) * (last - first + 1), 2)  
end
```

```
help?> my_sum  
search: my_sum  
  
my_sum  
=====
```

my_sum(first, last)

计算 [first, last] 之间的所有整数的求和

目录

1.常用操作命令和快捷键

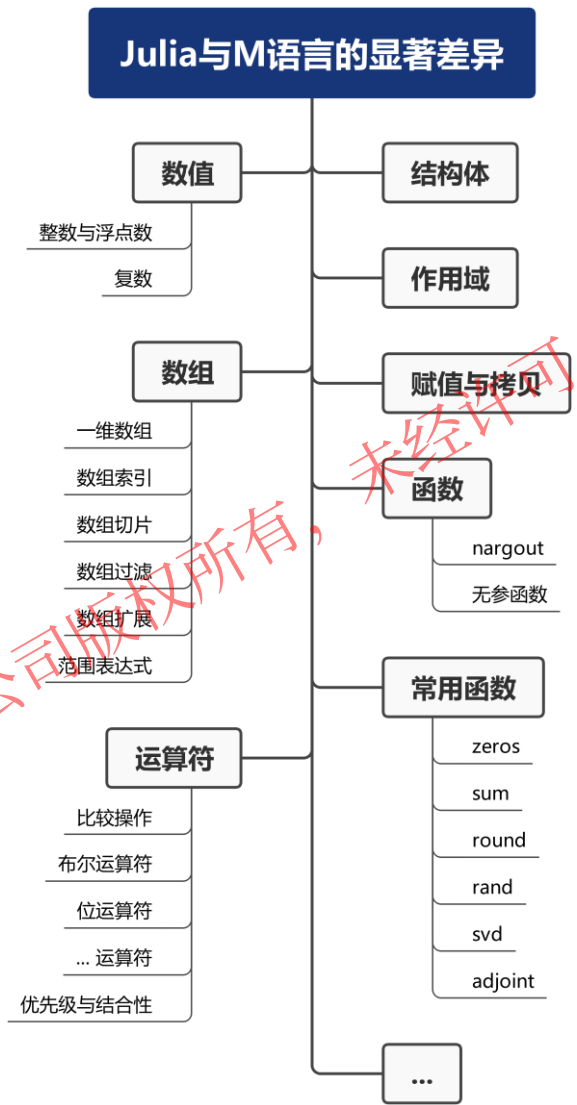
2. 矩阵分析

3. 脚本文件和函数文件

4. 与MATLAB的差异小结

4.1 与M语言的差异-概览

Julia 与 M 语言存在一些相似之处，但 Julia 不是 M 语言的克隆，两者存在显著差异



4.2 语法差异-数值

- 在Julia中，没有小数点的数字字面量会创建整数而不是浮点数
- 在Julia中，复数虚部为 `im`，而MATLAB中为 `i` 或 `j`

Julia

```
a=-1
typeof(a) # Int64

# 报错
2^a
#=
ERROR: DomainError with -1:
Cannot raise an integer x to a negative
power -1.
=#

# ok
2^float(a)

b = 1 + im
```

MATLAB

```
a=-1,
class(a) % 'double'

% ok
2^a

% 报错
a=int64(-1)
2^a
% Error using .^
% Integers can only be raised to positive
integral powers.

b = 1 + i
b = 1 + j
```

4.2 语法差异-数组

- **一维数组**: Julia 拥有真正的一维数组, 列向量的大小为 N , 而不是 $N \times 1$
- 数组索引: Julia 数组使用方括号 $A[i,j]$ 进行索引, MATLAB使用 $A(i,j)$ 进行索引
- 数组切片: Julia 对行向量($1 \times N$) 提取部分元素, 可返回向量

Julia

```
a = [1 2 3]
# 1×3 矩阵

b = [1,2,3]
# 向量

c = [1;2;3]
# 向量

d = [1;2;3;;]
# 3×1 矩阵
```

MATLAB

```
a = [1 2 3]
% 1×3 矩阵

b = [1,2,3]
% 1×3 矩阵

c = [1;2;3]
% 3×1 矩阵
```

4.2 语法差异-数组

- 一维数组: Julia 拥有真正的一维数组, 列向量的大小为 N , 而不是 $N \times 1$
- 数组索引: Julia 数组使用方括号 $A[i,j]$ 进行索引, MATLAB使用 $A(i,j)$ 进行索引
- **数组切片**: Julia 对行向量($1 \times N$) 提取部分元素, 可返回向量

Julia

```
x = [1 2 3 4 5 6]
# 1×6 Matrix{Int64}

# 提取部分元素, 返回向量
px = x[2:end]
# 5-element Vector{Int64}

# 该用法与MATLAB的x(2:end)或x(:,2:end)等价
px = x[:, 2:end]
# 1×5 Matrix{Int64}
```

MATLAB

```
x = [1 2 3 4 5 6]
% 1×6 矩阵

px = x(2:end)
% 1×5 矩阵

px = x(:,2:end)
% 1×5 矩阵
```

4.2 语法差异-数组

- **数组过滤**: 在 MATLAB 中, 删除不需要值的惯用方法是使用逻辑索引, 如表达式 $x(x>3)$ 或语句 $x(x>3) = []$ 来 in-place 修改 x
- 在 Julia 中, 可以使用高阶函数 `filter` 和 `filter!`, 或类似 $x[x .> 3]$ 等价写法

Julia

```
x = [1 3 5 7 9]
```

```
x[x .> 3]  
# [5, 7, 9]
```

```
x = x[x .<= 3]  
# x = [1, 3]
```

MATLAB

```
x = [1 3 5 7 9]
```

```
x(x>3)  
% [5 7 9]
```

```
x(x>3)=[]  
% x = [1 3]
```

4.2 语法差异-数组

- **数组扩展**: Julia 不会在赋值语句中自动增长数组
- 在 MATLAB 中, $a(4) = 3.2$ 可以创建数组 $a = [0 \ 0 \ 0 \ 3.2]$
- Julia 使用 `push!`、`append!` 来增长 Vector

Julia

```
# 针对向量
a = [0,0,0,3.2]
push!(a,7)
# [0.0, 0.0, 0.0, 3.2, 7.0]

# 针对矩阵
a = [0 0 0 3.2]
a = [a 7] # 等价于 a = hcat(a, 7)
# [0.0 0.0 0.0 3.2 7.0]
```

MATLAB

```
a = [0 0 0 3.2]
% 1x4 矩阵

a(5) = 7
% a = [0 0 0 3.2000 7.0000]
```

4.2 语法差异-数组

- **范围表达式**: 在 Julia 中, `a:b` 和 `a:b:c` 构造 `AbstractRange` 对象。使用 `collect(a:b)` 构造一个类似 MATLAB 中完整的向量
- 在大多数情况下, `AbstractRange` 对象将像普通数组一样运行, 但效率更高, 因为它是懒惰求值

Julia

```
x=1:2:10
# 1:2:9 (不展开, 懒惰求值, 效率高)

# 转换为向量
collect(x)
# [1, 3, 5, 7, 9]

# 创建100个元素的向量
y = LinRange(-5,5,100)

y1 = LinRange(-5,5,7)
# [-5.0, -3.33333, -1.66667, 0.0, 1.66667, 3.33333, 5.0]
```

MATLAB

```
x=1:2:10
% [1 3 5 7 9]

% 默认创建 1x100 矩阵
y = linspace(-5,5)

y1 = linspace(-5,5,7)
% [-5.0000 -3.3333 -1.6667 0 1.6667 3.3333 5.0000]
```

4.2 语法差异-结构体

- Julia的结构体 struct(不可变) 和 mutable struct(可变), 与MATLAB的 struct, 差异非常大
- 在Julia中, 对于 struct 不可变结构体, 创建后不可修改, 但性能高效

Julia

```
# 不可变结构体
struct Foo
    baz::Int
    qux::Float64
end

# 创建实例
foo = Foo(23, 1.5)

# 不允许修改
foo.qux = 2.0
# ERROR: setfield!:
immutable struct of type
Foo cannot be changed
```

Julia

```
# 可变结构体
mutable struct Bar
    baz::Int
    qux::Float64
end

# 创建实例
bar = Bar(23, 1.5);

# 允许修改
bar.qux = 2.0
```

MATLAB

```
% 方式1: 定义结构体对象
foo.baz = 23
foo.qux = 2.5

% 方式2: 结构体对象
bar = struct("baz", 23, "qux", 1.5)

% 允许修改
bar.qux = 2.0
```



4.2 语法差异-运算符

- **比较操作**: 在 Julia 中, 如果 A 和 B 是数组, 像 `A == B` 这样的逻辑比较运算符不会返回布尔值数组
- 相反地, 请使用 `A .== B`。对于其他的像是 `<`、`>` 的布尔运算符同理

Julia

```
a=[1,3,5,7,9]
b=[1,3,5,7,9]

# 数组判等
a == b
# true

# 数组元素判等 (向量化操作)
a .== b
# Bool[1, 1, 1, 1, 1]

# ok: 需要向量化操作
a .> 3
# Bool[0, 0, 1, 1, 1]
```

MATLAB

```
a=[1,3,5,7,9]
b=[1,3,5,7,9]

% 数组元素判等
a == b
% 1x5 logical array
% 1 1 1 1 1

% 数组判等
isequal(a, b)
% true

a > 3
% 1x5 logical array
% 0 0 1 1 1
```

4.2 语法差异-运算符

- **布尔运算符**: 在Julia中, 运算符采用 **!** (否定)、**&&** (短路与)、**||** (短路或) 进行布尔运算, 分别与MATLAB中的 **~**、**&&**、**||** 等价
- 注意, MATLAB中的 **&** 与 **|** 表示无短路功能的逻辑运算, 不是位运算符

Julia

```
A = false  
  
# 逻辑 not  
!A  
# true
```

MATLAB

```
A = false  
  
% 逻辑 not  
~A
```

4.2 语法差异-运算符

- **位运算符**: 在 Julia 中, 运算符 `&`、`|` 和 `⊕` (`xor`) 进行按位操作, 分别与MATLAB中的 `bitand`、`bitor`、`bitxor` 等价

Julia

```
# 按位取反
~4 # -5

# 按位与
4 & 10 # 0
4 & 12 # 4

# 按位或
4 | 10 # 14
4 | 1 # 5

# 按位异或
xor(4,7) # 3
```

MATLAB

```
% 按位补码
bitcmp(int8(4)) % int8(-5)

% 按位与
bitand(int8(4), int8(10)) % int8(0)
bitand(int8(4), int8(12)) % int8(4)

% 按位或
bitor(int8(4), int8(10)) % int8(14)
bitor(int8(4), int8(1)) % int8(5)

% 按位异或
bitxor(int8(4), int8(7)) % int8(3)
```

4.2 语法差异-运算符

- **...运算符**：在Julia中，...运算符有两种用法：一是将多个参数组合成一个参数，二是将一个参数分解成多个不同参数；但在MATLAB中，... 表示续行符
- Julia没有续行符，Julia中不完整的表达式会自动延续到下一行

Julia

```
# 用法1: 将多个参数组合成一个参数
function printargs(args...)
    println(typeof(args))
    for (i, arg) in enumerate(args)
        println("Arg # $i = $arg")
    end
end

printargs(10, 20, 30)
#=
Tuple{Int64, Int64, Int64}
Arg #1 = 10
Arg #2 = 20
Arg #3 = 30
=#
```

Julia

```
# 用法2: 将一个参数分解成多个不同参数
function threeargs(a, b, c)
    println("a = $a::$(typeof(a))")
    println("b = $b::$(typeof(b))")
    println("c = $c::$(typeof(c))")
end

x = [1, 2, 3]
threeargs(x...)
#=
a = 1::Int64
b = 2::Int64
c = 3::Int64
=#
```

Julia

```
# Julia没有续行符
y = 1 +
    2 +
    4
# 7
```

MATLAB

```
% ...表示续行符
y = 1 +...
    2 +...
    4
% 7
```



4.2 语法差异-运算符

- 运算符优先级**: 在 Julia 中, a^b^c 被认为是 $a^{(b^c)}$ 而在 MATLAB 中它是 $(a^b)^c$, 这是因为 Julia 的 $^$ 是右结合。建议使用 $()$ 指定优先级

Julia

```
# Julia中, ^是右结合  
4^3^2 # 等价于 4^(3^2)  
# 262144
```

```
(4^3)^2  
# 4096
```

MATLAB

```
% M语言中, ^是左结合  
4^3^2 % 等价于 (4^3)^2  
% 4096
```

```
4^(3^2)  
% 262144
```

4.2 语法差异-作用域

- 在 Julia 中，每个模块有自身的全局作用域/命名空间，而在 MATLAB 中只有一个全局作用域
- 在 Julia 中，用户难以理解的问题是 **for 作用域**，参见 Syslab 帮助文档中的【[Syslab>Julia语言概览>变量作用域](#)】

Julia REPL

```
# 在交互式环境下，如REPL
s = 0
for i = 1:10
    s += i
end
s # 55
```

Julia 文件

```
# 在非交互环境下运行，如文件
s = 0
for i = 1:10
    s += i
end
# ERROR: UndefVarError: `s`
# not defined
```

MATLAB

```
% 在文件中执行，与在命令行窗口
% 执行，表现一致
s = 0
for i = 1:10
    s = s+i;
end
s
% 55
```



4.2 语法差异-赋值与拷贝

- MATLAB 使用按值传递 (Call by value) , 防止数组被意外地篡改, 但也会导致不必要的数组拷贝
- Julia 与 Python、Ruby等语言, 都采用共享调用技术 (Call by sharing) , 详情参见 Syslab 帮助文档中的【Syslab>Julia语言概览>赋值与拷贝】

Julia

```
# x和y指向同一块内存
x=[1,2,3]
y=x
x[1] = 10
y # [10,2,3]
```

Julia

```
# 数组拷贝
x = [1,2,3]
y = copy(x)
x[1] = 10
y # [1,2,3]
```

MATLAB

```
% 拷贝
x=[1,2,3]
y=x
x(1) = 10
y
% [1,2,3]
```



4.2 语法差异-函数

- 在 Julia 中，能返回多个值并将其赋值为元组，例如 $(a, b) = (1, 2)$ 或 $a, b = 1, 2$
- 在 Julia 中不存在 MATLAB 的 `nargout`，它通常在 MATLAB 中用于根据返回值的数量执行可选工作。取而代之的是，用户可以使用可选参数和关键字参数来实现类似的功能

Julia

```
function addme2(a,b)
    result = a+b
    absResult = abs(result)
    return result, absResult
end
```

```
addme2(1,-3)
# 返回元组 (-2, 2)
```

```
x, = addme2(1,-3)
# x = -2
```

```
x, y = addme2(1,-3)
# x = -2, y = 2
```

MATLAB

```
% 文件1
function [result,absResult] = addme2(a,b)
    result = a+b
    if nargin > 1
        absResult = abs(result);
    end
end
```

```
% 文件2 或 在命令行窗口输入
addme2(1,-3)
% 返回 -2
```

```
[x,y]=addme2(1,-3)
% x = -2, y = 2
```

4.2 语法差异-函数

- 在 Julia 中，调用无参数的函数时必须使用小括号，例如 rand()
- 在 MATLAB 中，调用无参数的函数时可以不带小括号

Julia

```
# 函数变量
rand
# rand (generic function with 81 methods)

# 函数调用
rand()
# 0.3125316543482983
```

MATLAB

```
% rand 与 rand() 等价
rand % 0.8147
rand() % 0.9058
```

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

感谢聆听